- RAG Learning

2.1 Algorithms

OCR GCSE Computer Science (J277) — Paper 2

2.1.1 Computational Thinking

> **Computational thinking** is a structured problem-solving approach that involves breaking down complex problems into manageable parts, identifying the key information, and designing step-by-step solutions. It is not about thinking like a computer — it is about thinking in a way that allows a computer (or human) to solve the problem systematically.

Computational thinking involves three key techniques that are used together:

Technique	Definition	Example
Decomposition	Breaking a complex problem into smaller, more manageable **sub-problems** that can be solved independently. Each sub-problem is simpler to understand, implement, test, and debug. Different team members can work on different sub-problems simultaneously.	Building a game: decompose into sub-problems — graphics rendering, user input handling, scoring system, sound effects, level design, save/load functionality, multiplayer networking. Each can be developed and tested independently before being integrated.
Abstraction	Removing unnecessary detail and focusing only on the **relevant** information needed to solve the problem. Hides complexity behind a simpler representation. Identifies what IS important and filters out what is NOT.	A London Underground map abstracts away the actual geographic positions of stations, the depth of tunnels, and the distances between stops. It shows only what a passenger needs: station names, lines, and connections. A weather forecast shows temperature and

		conditions, abstracting away complex atmospheric models.
Algorithmic thinking	Designing a precise, unambiguous, **step-by-step solution** (algorithm) that can be followed mechanically to solve the problem. Steps must be in the correct order, cover all possible cases (including edge cases), and terminate (finish).	Finding the largest number in a list: (1) Assume the first number is the largest. (2) Compare it to each subsequent number. (3) If a larger number is found, update the record. (4) After checking all numbers, the recorded value is the answer. This works for any list of any length.

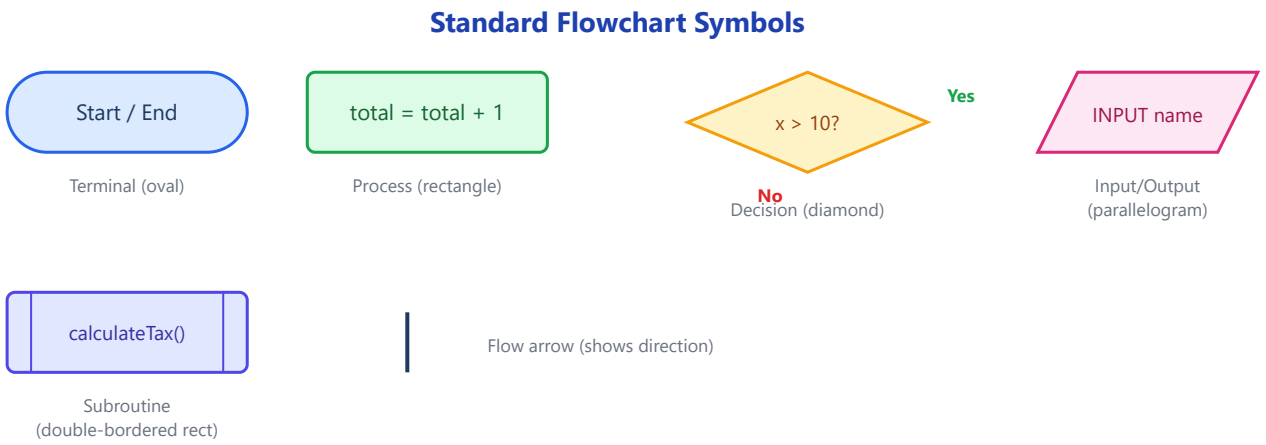
Applying Computational Thinking — Worked Example

```
Problem: "Build a school registration system"

DECOMPOSITION — break into sub-problems:
  1. User login (authentication)
  2. Class list management (add/remove students)
  3. Taking the register (mark present/absent/late)
  4. Calculating attendance percentages
  5. Generating reports for parents/teachers
  6. Storing data in a database

ABSTRACTION — what details matter?
  Important: student names, dates, attendance status, class groups
  Not important: student height, favourite colour, shoe size
  We model only the information relevant to registration.

ALGORITHMIC THINKING — step-by-step solution for "calculate attendance":
  1. Count total number of school days in the period
  2. Count number of days the student was marked present
  3. Attendance % = (days present / total days) x 100
  4. If attendance < 90%, flag as concern
  5. Display result
```

2.1.2 Representing Algorithms

Algorithms can be represented in three main ways: **flowcharts** (visual diagrams), **pseudocode** (structured English-like text), and **program code** (actual programming language). The exam uses flowcharts and OCR-style pseudocode.

Flowchart Symbols



Symbol	Shape	Meaning	Example
Terminal	Oval / rounded rectangle	Start or End of the algorithm	"Start", "End", "Stop"
Process	Rectangle	An action, calculation, or assignment	"total = total + price", "count = 0"
Decision	Diamond	A yes/no (true/false) question that creates a branch. Has two exit paths.	"Is score > 50?", "Is x == 0?"
Input / Output	Parallelogram	Reading input from user or displaying output	"INPUT name", "OUTPUT total"
Flow arrow	Arrow	Shows the direction of flow between steps	Connects all symbols in sequence

Subroutine	Rectangle with double side borders	A call to a separate subroutine (procedure or function)	"CALL calculateTax()"

Pseudocode Constructs (OCR Style)

Pseudocode is a structured way to describe an algorithm using English-like syntax. It is not a real programming language — there is no compiler or interpreter for it. The purpose is to express the logic clearly without worrying about language-specific syntax.

```
// ── SELECTION (IF-ELSE) ──────────────────────────────────
// Used when the program must make a decision

if score >= 50 then
    print("Pass")
else
    print("Fail")
endif

// Multiple conditions using ELSEIF:
if score >= 70 then
    print("Distinction")
elseif score >= 50 then
    print("Merit")
elseif score >= 40 then
    print("Pass")
else
    print("Fail")
endif

// SWITCH / CASE — for testing one variable against specific values:
switch day:
    case "Monday":
        print("Start of week")
    case "Friday":
        print("Nearly weekend!")
    default:
        print("Midweek")
endswitch

// ── COUNT-CONTROLLED LOOP (FOR) ──────────────────────────
// Runs a KNOWN number of times
```

```
for i = 1 to 10
    print(i)
next i

// Python equivalent:
// for i in range(1, 11):
//     print(i)

// ── CONDITION-CONTROLLED LOOP (WHILE) ──────────────────────
// Checks condition BEFORE each iteration — may run 0 times

while password != "abc123"
    password = input("Enter password: ")
endwhile

// Python equivalent:
// while password != "abc123":
//     password = input("Enter password: ")

// ── POST-CONDITION LOOP (DO-UNTIL) ─────────────────────────
// Checks condition AFTER each iteration — runs AT LEAST once

do
    age = int(input("Enter age (0-120): "))
until age >= 0 AND age <= 120

// Python equivalent (no do-until in Python, use while True + break):
// while True:
//     age = int(input("Enter age (0-120): "))
//     if 0 <= age <= 120:
//         break
```

Trace Tables

A **trace table** tracks the values of variables at each step of an algorithm's execution. It is an essential debugging and verification tool and is frequently required in exam questions. Create a column for each variable and a row for each step that changes a value.

```
// Algorithm to trace:
x = 3
for i = 1 to 4
```

```
    x = x + i
next i
print(x)
```

Step	i	x	Calculation
Init	—	3	x = 3
1	1	4	x = 3 + 1 = 4
2	2	6	x = 4 + 2 = 6
3	3	9	x = 6 + 3 = 9
4	4	13	x = 9 + 4 = 13

Output: 13

More Complex Trace Table Example

```
// Algorithm with selection inside a loop:
total = 0
for i = 1 to 5
    if i MOD 2 == 0 then
        total = total + i
    endif
next i
print(total)
```

Step	i	i MOD 2	Condition	total
Init	—	—	—	0
1	1	1	False (skip)	0
2	2	0	True (add)	2
3	3	1	False (skip)	2

© RAG Learning · Not for redistribution

4	4	0	True (add)	6
5	5	1	False (skip)	6

Output: 6 (sum of even numbers: 2 + 4 = 6)

> **Exam Tip:** In trace table questions, include **intermediate columns** for sub-expressions (like `i MOD 2`) to show your working. This earns method marks even if your final answer has an error. Always add the initial values as the first row. For WHILE loops, remember to show the final failed condition check that causes the loop to exit.

2.1.3 Searching Algorithms

Linear Search

> **Linear search** checks each item in the list **one by one from the start** until the target is found or the end of the list is reached. It works on both sorted and unsorted data. It is the simplest search algorithm.

```
// OCR Pseudocode:
function linearSearch(list, target)
    for i = 0 to length(list) - 1
        if list[i] == target then
            return i           // found at index i
        endif
    next i
    return -1                  // not found (searched entire list)
endfunction

# Python:
def linear_search(data, target):
    for i in range(len(data)):
        if data[i] == target:
```

```
            return i
    return -1
```

Linear Search — Step-by-Step Trace

```
Searching for 25 in [3, 7, 12, 19, 25, 31, 44, 58]

  Index 0: 3  — 3 == 25? No.  Continue.
  Index 1: 7  — 7 == 25? No.  Continue.
  Index 2: 12 — 12 == 25? No. Continue.
  Index 3: 19 — 19 == 25? No. Continue.
  Index 4: 25 — 25 == 25? YES! FOUND at index 4.

  Result: Found at index 4, after 5 comparisons.

Searching for 20 in [3, 7, 12, 19, 25, 31, 44, 58]

  Index 0: 3  — No.   Index 1: 7  — No.   Index 2: 12 — No.
  Index 3: 19 — No.   Index 4: 25 — No.   Index 5: 31 — No.
  Index 6: 44 — No.   Index 7: 58 — No.   End of list reached.

  Result: NOT FOUND, after 8 comparisons (checked every element).
```

Feature	Detail
Data requirement	Works on **unsorted AND sorted** data — no preprocessing needed
Best case	O(1) — target is the first element (1 comparison)
Worst case	O(n) — target is last or not present (must check every element)
Average case	O(n/2) which simplifies to O(n)
Advantages	Simple to implement and understand; works on unsorted data; no pre-processing; efficient for small lists
Disadvantages	Very slow for large datasets (must check every element in worst case); inefficient compared to binary search on sorted data

Binary Search

Binary search repeatedly **halves the search space** by comparing the target to the middle element. If the target is smaller, search the left half; if larger, search the right half. The data **must be sorted first**. Each comparison eliminates half of the remaining elements.

```
// OCR Pseudocode:
function binarySearch(list, target)
    low = 0
    high = length(list) - 1
    while low <= high
        mid = (low + high) DIV 2
        if list[mid] == target then
            return mid         // found
        elseif list[mid] < target then
            low = mid + 1      // target is in right half
        else
            high = mid - 1     // target is in left half
        endif
    endwhile
    return -1                  // not found
endfunction

# Python:
def binary_search(data, target):
    low, high = 0, len(data) - 1
    while low <= high:
        mid = (low + high) // 2
        if data[mid] == target:
            return mid
        elif data[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1
```

Binary Search — Step-by-Step Trace

```
Searching for 25 in [3, 7, 12, 19, 25, 31, 44, 58]
                     0  1   2   3   4   5   6   7

Step 1: low=0, high=7, mid=(0+7)DIV2 = 3
        list[3] = 19.   25 > 19, so search RIGHT half.
        low = 3 + 1 = 4.
        Remaining search space: [25, 31, 44, 58]

Step 2: low=4, high=7, mid=(4+7)DIV2 = 5
        list[5] = 31.   25 < 31, so search LEFT half.
        high = 5 - 1 = 4.
        Remaining search space: [25]

Step 3: low=4, high=4, mid=(4+4)DIV2 = 4
        list[4] = 25.   25 == 25. FOUND at index 4!

Result: Found in only 3 comparisons (vs 5 for linear search).
        On larger lists the difference is dramatic.
```

Binary Search — Not Found Example

```
Searching for 20 in [3, 7, 12, 19, 25, 31, 44, 58]

Step 1: low=0, high=7, mid=3, list[3]=19.  20 > 19 → low=4
Step 2: low=4, high=7, mid=5, list[5]=31.  20 < 31 → high=4
Step 3: low=4, high=4, mid=4, list[4]=25.  20 < 25 → high=3
Step 4: low=4, high=3.  low > high → STOP.  NOT FOUND.

Result: Not found after 3 comparisons (vs 8 for linear search).
```

Feature	Detail
Data requirement	**Must be sorted first** — this is critical and must always be stated
Best case	O(1) — target is the exact middle element
Worst case	O(log n) — extremely efficient even for huge datasets

Advantage	Very fast for large sorted datasets. Searching 1 billion items takes at most ~30 comparisons (log2 of 1 billion).
Disadvantage	Data must be sorted first (sorting costs O(n log n) time); more complex to implement; less efficient than linear search for very small lists

Comparison: Linear vs Binary Search

List Size	Linear Search (worst case)	Binary Search (worst case)
10 items	10 comparisons	4 comparisons
1,000 items	1,000 comparisons	10 comparisons
1,000,000 items	1,000,000 comparisons	20 comparisons
1,000,000,000 items	1,000,000,000 comparisons	30 comparisons

Exam Tip: **Binary search requires sorted data** — always state this explicitly in your answer. If the data is not sorted, you must either sort it first (which takes time) or use linear search. For trace questions, show low, high, and mid at each step, and clearly show which half is eliminated. You do NOT need to memorise algorithm code — only be able to apply and trace algorithms on a given dataset.

2.1.4 Sorting Algorithms

Bubble Sort

Bubble sort repeatedly compares **adjacent pairs** of elements and swaps them if they are in the wrong order. After each complete pass through the list, the next largest unsorted element "bubbles up" to its correct position at the end. Passes repeat until no swaps are made in a complete pass (meaning the list is fully sorted).

```
// OCR Pseudocode:
procedure bubbleSort(list)
    n = length(list)
    do
        swapped = false
        for i = 0 to n - 2
            if list[i] > list[i+1] then
                // Swap the two elements
                temp = list[i]
                list[i] = list[i+1]
                list[i+1] = temp
                swapped = true
            endif
        next i
        n = n - 1    // last element is now in correct position
    until swapped == false
endprocedure

# Python:
def bubble_sort(data):
    n = len(data)
    while True:
        swapped = False
        for i in range(n - 1):
            if data[i] > data[i + 1]:
                data[i], data[i + 1] = data[i + 1], data[i]
                swapped = True
        n -= 1
        if not swapped:
            break
```

Bubble Sort — Complete Trace

```
Sorting: [5, 3, 8, 1, 9, 2]

Pass 1 (compare all adjacent pairs):
  Compare 5,3 → 5>3 → SWAP → [3, 5, 8, 1, 9, 2]
  Compare 5,8 → 5<8 → no swap
  Compare 8,1 → 8>1 → SWAP → [3, 5, 1, 8, 9, 2]
  Compare 8,9 → 8<9 → no swap
  Compare 9,2 → 9>2 → SWAP → [3, 5, 1, 8, 2, 9]
```

© RAG Learning · Not for redistribution

```
  ✓ 9 is now in its correct final position (largest bubbles to end)
  Swaps made: YES, continue.

Pass 2 (ignore last position — 9 is sorted):
  Compare 3,5 → no swap
  Compare 5,1 → SWAP → [3, 1, 5, 8, 2, 9]
  Compare 5,8 → no swap
  Compare 8,2 → SWAP → [3, 1, 5, 2, 8, 9]
  ✓ 8 is now in its correct position.

Pass 3 (ignore last 2 positions):
  Compare 3,1 → SWAP → [1, 3, 5, 2, 8, 9]
  Compare 3,5 → no swap
  Compare 5,2 → SWAP → [1, 3, 2, 5, 8, 9]
  ✓ 5 is in position.

Pass 4:
  Compare 1,3 → no swap
  Compare 3,2 → SWAP → [1, 2, 3, 5, 8, 9]
  ✓ 3 is in position.

Pass 5:
  Compare 1,2 → no swap
  No swaps made in this pass → list is SORTED! STOP.
  Final result: [1, 2, 3, 5, 8, 9]
```

Merge Sort

Merge sort uses a **divide and conquer** strategy. It works in two phases: (1) **Split** — recursively divide the list in half until each sub-list has one element (a single element is trivially sorted). (2) **Merge** — repeatedly merge pairs of sub-lists back together in sorted order. It is much more efficient than bubble sort for large datasets.

```
// OCR Pseudocode (simplified):
function mergeSort(list)
    if length(list) <= 1 then
        return list    // base case: single element is already sorted
    endif
    mid = length(list) DIV 2
    left = mergeSort(list[0..mid-1])     // sort left half
```

```
    right = mergeSort(list[mid..end])     // sort right half
    return merge(left, right)            // merge the two sorted halves
endfunction

# Python:
def merge_sort(data):
    if len(data) <= 1:
        return data
    mid = len(data) // 2
    left = merge_sort(data[:mid])
    right = merge_sort(data[mid:])
    return merge(left, right)

def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result
```

Merge Sort — Complete Trace

```
Sorting: [5, 3, 8, 1, 9, 2]

SPLIT PHASE (divide until single elements):
  [5, 3, 8, 1, 9, 2]
  ├── [5, 3, 8]              [1, 9, 2]
  ├── [5] [3, 8]             [1] [9, 2]
  └── [5] [3] [8]            [1] [9] [2]

MERGE PHASE (merge pairs in sorted order):
  Merge [3] + [8] → compare 3,8 → [3, 8]
  Merge [5] + [3,8] → compare 5,3→take 3; compare 5,8→take 5; take 8 → [3, 5, 8]

  Merge [9] + [2] → compare 9,2 → [2, 9]
```

```
  Merge [1] + [2,9] → compare 1,2→take 1; take 2; take 9 → [1, 2, 9]

  Final merge [3,5,8] + [1,2,9]:
    Compare 3,1 → take 1 → [1]
    Compare 3,2 → take 2 → [1, 2]
    Compare 3,9 → take 3 → [1, 2, 3]
    Compare 5,9 → take 5 → [1, 2, 3, 5]
    Compare 8,9 → take 8 → [1, 2, 3, 5, 8]
    Take remaining 9 →       [1, 2, 3, 5, 8, 9]

  Result: [1, 2, 3, 5, 8, 9]
```

Insertion Sort

Insertion sort builds the sorted list one element at a time. It takes each element from the unsorted portion and **inserts it into its correct position** within the already-sorted portion, shifting elements to the right as needed. It works like sorting playing cards in your hand.

```
// OCR Pseudocode:
procedure insertionSort(list)
    for i = 1 to length(list) - 1
        key = list[i]            // element to insert
        j = i - 1
        while j >= 0 AND list[j] > key
            list[j+1] = list[j]  // shift element right
            j = j - 1
        endwhile
        list[j+1] = key          // insert into correct position
    next i
endprocedure

# Python:
def insertion_sort(data):
    for i in range(1, len(data)):
        key = data[i]
        j = i - 1
        while j >= 0 and data[j] > key:
            data[j + 1] = data[j]
```

```
            j -= 1
        data[j + 1] = key
```

Insertion Sort — Complete Trace

```
Sorting: [5, 3, 8, 1, 9, 2]
         ↑ sorted portion starts with first element

Step 1: Insert 3 into sorted [5]
  3 < 5, shift 5 right, insert 3 at start
  Result: [3, 5, | 8, 1, 9, 2]

Step 2: Insert 8 into sorted [3, 5]
  8 > 5, so insert at end (no shifting needed)
  Result: [3, 5, 8, | 1, 9, 2]

Step 3: Insert 1 into sorted [3, 5, 8]
  1 < 8, shift 8; 1 < 5, shift 5; 1 < 3, shift 3; insert 1 at start
  Result: [1, 3, 5, 8, | 9, 2]

Step 4: Insert 9 into sorted [1, 3, 5, 8]
  9 > 8, insert at end
  Result: [1, 3, 5, 8, 9, | 2]

Step 5: Insert 2 into sorted [1, 3, 5, 8, 9]
  2 < 9, shift; 2 < 8, shift; 2 < 5, shift; 2 < 3, shift; 2 > 1, insert after 1
  Result: [1, 2, 3, 5, 8, 9]
```

Sorting Algorithm Comparison

Algorithm	Best Case	Worst Case	Memory	Key Characteristic
Bubble sort	O(n) — already sorted	O(n^2) — reverse sorted	O(1) in-place	Simplest; least efficient for large data; easy to detect when list is already sorted (no swaps = stop); good for teaching
Merge sort	O(n log n)	O(n log n) — always	O(n) extra	Most efficient and consistent; divide and conquer; requires extra memory for

				merging; best for large datasets
Insertion sort	O(n) — already sorted	O(n^2) — reverse sorted	O(1) in-place	Good for small datasets or nearly-sorted data; sorts in-place; efficient when inserting into an already-sorted list

Exam Tip: You do not need to memorise code for these algorithms. You DO need to be able to **trace through** each algorithm on a given dataset, showing the state of the list after each pass/step. For merge sort, show **both** the complete split phase AND the merge phase with each comparison. For bubble sort, show each comparison and swap within each pass, and clearly indicate when a pass has no swaps (which triggers termination). For insertion sort, show the sorted and unsorted portions at each step.

2.1.5 Big-O Notation and Algorithm Efficiency

Big-O notation describes how the time (or space) an algorithm takes **grows** as the size of its input (n) increases. It expresses the *worst-case* relationship between input size and the number of steps required. It is a way of comparing algorithms independently of hardware speed or programming language — it focuses purely on how the algorithm scales.

The key question Big-O answers is: "If I double the number of items, what happens to the time taken?" The notation uses **n** to represent the number of items in the input.

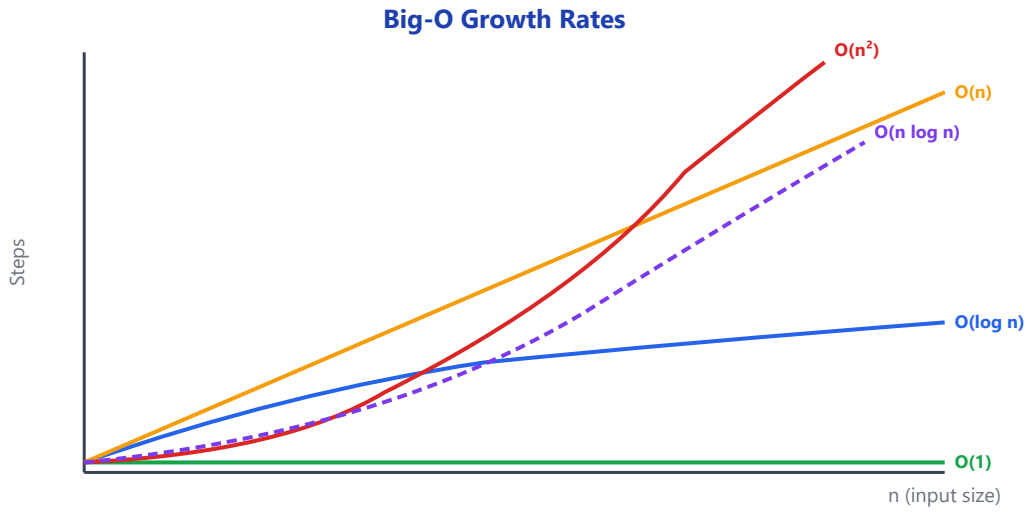
Notation	Name	Meaning	Example Algorithm	Behaviour when n doubles
O(1)	Constant time	The algorithm always takes the **same amount of time** regardless of input size. It does not depend on n at all.	Accessing an array element by index (`scores[3]`); checking if a stack is empty; looking up a value in a hash table.	Time stays the same. Doubling n has no effect.

O(log n)	Logarithmic time	The number of steps grows very slowly — each step **halves** the remaining problem. Very efficient for large inputs.	Binary search. With 1 billion items, takes only ~30 steps ($\log_2(1{,}000{,}000{,}000) \approx 30$).	Time increases by just 1 step. Doubling from 1M to 2M items adds only 1 more comparison.
O(n)	Linear time	The number of steps grows **proportionally** to the input size. Double the items = double the time.	Linear search; traversing every element of a list; reading all elements of an array once.	Time doubles. n items → n steps.
O(n log n)	Linearithmic time	Slightly worse than linear — common for efficient sorting algorithms that divide the problem into parts.	Merge sort; efficient sorting algorithms generally.	Time slightly more than doubles — highly efficient for sorting.
O(n^2)	Quadratic time	The number of steps grows as the **square** of the input size. Double the items = four times the time. Gets very slow quickly.	Bubble sort; insertion sort; any algorithm with two nested loops each iterating through all n items.	Time quadruples. 10 items → 100 steps; 100 items → 10,000 steps; 1,000 items → 1,000,000 steps.

Visual Comparison — Growth Rates

```
Steps taken for different input sizes (n):

 n    | O(1) | O(log n) | O(n)  | O(n log n) | O(n²)
------|------|----------|-------|------------|----------
   10 |    1 |        4 |    10 |         40 |       100
```

```
  100 |     1 |         7 |    100 |         700 |     10,000
1,000 |     1 |        10 |   1000 |      10,000 | 1,000,000
1,000,000 | 1 |    20 | 1,000,000 | 20,000,000 | 1,000,000,000,000

Conclusion:
  O(1)      – ideal (never gets slower)
  O(log n)  – excellent (binary search)
  O(n)      – acceptable (linear search)
  O(n log n)– good (merge sort)
  O(n²)     – poor for large n (bubble/insertion sort)
```



Lower = more efficient. O(1) is best; O(n^2) is worst shown here.

Big-O for Algorithms You Have Studied

Algorithm	Best Case	Worst Case	Why?
Linear search	O(1) — target is first element	**O(n)** — must check every element	In the worst case, the target is at the end or not in the list at all, so every element is checked once.
Binary search	O(1) — target is the middle element	**O(log n)** — each step halves the search space	With 8 items: max 3 steps. With 16 items: max 4 steps. Each doubling of n adds only 1 step. Requires sorted data.
Bubble sort	O(n) — already sorted (detect	**O(n^2)** — two nested passes through all	For each of n elements, a pass through up to n-1 pairs is needed. n

	with "no swaps" flag)	elements	× n = n^2.
Insertion sort	O(n) — already sorted (each element already in place)	**O(n^2)** — reverse sorted (every element must shift past all others)	Same reason as bubble sort — can require nested iterations.
Merge sort	O(n log n)	**O(n log n)** — always consistent	The divide phase takes log n levels; merging takes O(n) at each level. n × log n total. Consistent regardless of input order.

Choosing the Right Algorithm

```
Q: "Should I use linear or binary search on a list of 1 million names?"

Linear search: up to 1,000,000 comparisons in the worst case.
Binary search: up to 20 comparisons (log₂(1,000,000) ≈ 20).

Binary search is roughly 50,000x faster for large sorted datasets.
BUT: binary search requires the list to be sorted first.
Sorting costs O(n log n) — about 20,000,000 steps for 1 million items.
If you only search once, sorting first + binary searching is SLOWER
than just doing a linear search.
If you search many times, sorting once and binary searching repeatedly
is far more efficient overall.

Rule of thumb:
  Small list (< 20 items): linear search — simpler, overhead not worth it
  Large sorted list, many searches: binary search — dramatically faster
  Large unsorted list, one search: linear search — sorting first costs more
```

Exam Tip: Big-O tells you the **worst-case scaling behaviour**, not the exact number of steps. O(n^2) does not mean "n squared steps" exactly — it means "the steps grow proportionally to n^2". The most important facts to know: **O(1)** = constant (array index lookup); **O(log n)** = binary search; **O(n)** = linear search; **O(n log n)** = merge sort; **O(n^2)** = bubble/insertion sort. In exam answers, always state the Big-O of both algorithms when comparing them, and explain what it

means for performance at scale: "Binary search is O(log n) so for 1 million items it requires only about 20 comparisons, whereas linear search is O(n) and may require up to 1 million comparisons."